Applied Programming

Week 7: Review

Connecting concepts and coding skills from Weeks 1–6

School of Mechanical Engineering, Soongsil University
2026-2 Applied Programming

A useful program does more than produce a number. It gives inputs a clear meaning, performs a calculation, checks its result and keeps related data consistent. This week revisits the tools used to do those jobs. The code, data and worked examples are reused from Weeks 1–6. This is review, not a new Assignment or a preview of the examination questions.

1 Run, inspect and save a program

Review connection: Week 1, Sections 1 and 4–10.

A **program** is a set of instructions. The Python **interpreter** executes those instructions. A **virtual environment** keeps a project's interpreter and installed packages separate from other projects. Activating an environment selects which Python and packages a command uses. It does not change the folder or correct errors in a calculation.

Prepare the supplied review files

Extract `week07_review_examples.zip`. Work in its `week07_review` folder, outside your earlier assignment repositories. Keep the supplied `week02`, `week03`, `week04` and `week05` folders together. The commands shown run the supplied examples, not unfinished functions in your earlier assignment repositories. Use the existing Week 6 environment. A GPU and a new package installation are not needed.

Run – Anaconda Prompt

Replace the example folder below with the folder you extracted.

```
conda activate applied-programming-w06
cd /d "C:\your-course-folder\week07_review"
dir
python -c "import sys; print(sys.executable)"
```

The directory listing must include the week folders. The interpreter path must point to the activated environment. On macOS or Linux, use `cd "/your/folder/week07_review"` and `ls` instead of `cd /d` and `dir`. The Python commands are the same. Enter commands in a terminal, not after Python's `>>>` prompt. Each section names the working folder and original file. Each Output box shows the result of the adjacent code, not the output of an unrelated command.

An environment check verifies the tools available to run a program. A calculation test checks a result against a stated expectation. An environment can therefore work correctly while a calculation test fails. In the following sections, read the input and expected meaning before interpreting a test result.

Files, imports and saved revisions

A **module** is a Python file that can be imported. A **package** groups related modules. As in Week 1, `if __name__ == "__main__":` separates direct execution from importing a module. Code inside that block runs when the file is executed directly.

Git records versions of files. `git diff` shows unstaged edits, `git add` stages the current content, and `git commit` records the staged content locally. `git push` transfers commits to a remote repository. Saving in an editor, committing and pushing are different actions. If a file changes after staging, stage the new change again to include it in the next commit. This review ZIP does not need to become a new repository.

2 Write functions with checkable results

Review connection: Week 2, Sections 2.1–2.3.

The following examples reuse Week 2's coordinates, functions and checks. The supplied `week02/learning_geometry.py` contains the complete practice definitions from that Reading. It does not import unfinished Assignment functions. The original interactive calls are collected, without changing their calculations, in the runnable files named below. Each Output box shows the result of the adjacent code.

2.1 Follow the value from input to return

A **function** names a reusable calculation or operation. Its parameters name the inputs inside the function. **Arguments** are the values supplied by a call. The **caller** is the code making the call. Assignment stores the returned value in a name, while `print` displays it.

The original 3–4–5 calculation keeps the intermediate values easy to follow. Coordinates have the same course coordinate units used in Week 2.

Run – Anaconda Prompt, from week07_review

```
cd week02
python values_and_calls.py
```

Code – values_and_calls.py, first calculation

```
start = (0.0, 0.0)
goal = (3.0, 4.0)

dx = goal[0] - start[0]
dy = goal[1] - start[1]
squared_length = dx * dx + dy * dy

def finish_distance(value):
    return value**0.5

distance = finish_distance(squared_length)
print(distance)
```

Output

```
5.0
```

The differences are `dx=3.0` and `dy=4.0`. Their squared sum is `25.0`. Calling `finish_distance(squared_length)` returns `5.0`, which is assigned to `distance`. The following `print` displays that value. The returned value and the displayed text have different roles.

2.2 Keep points and their order in containers

A **tuple** records an ordered pair whose entries cannot be reassigned. A **list** records an ordered sequence that can be extended or edited. Here, each tuple is one point and the list is the path. That meaning comes from the program, not from the container alone.

Code – the same file, original waypoint list

```
start = (0.0, 0.0)
corner = (3.0, 0.0)
goal = (3.0, 4.0)
waypoints = [start, corner]
waypoints.append(goal)

x_goal, y_goal = goal
first_two = waypoints[:2]
print(waypoints[1], first_two, x_goal, y_goal)
```

Output

```
(3.0, 0.0) [(0.0, 0.0), (3.0, 0.0)] 3.0 4.0
```

Indexing starts at zero. `waypoints[1]` selects the second waypoint. The slice `waypoints[:2]` includes indices 0 and 1 but excludes the stop index 2. Unpacking assigns the two entries in `goal` to `x_goal` and `y_goal`. `append` adds one waypoint at the end of the list.

Code – the same file, adjacent waypoint pairs

```
segments = []
for p0, p1 in zip(waypoints[:-1], waypoints[1:]):
    displacement = (p1[0] - p0[0], p1[1] - p0[1])
    segments.append(displacement)

for index, displacement in enumerate(segments):
    print(index, displacement)

segments_comp = [
    (p1[0] - p0[0], p1[1] - p0[1])
    for p0, p1 in zip(waypoints[:-1], waypoints[1:])
]
```

Output

```
0 (3.0, 0.0)
1 (0.0, 4.0)
```

The two slices pair the first waypoint with the second and then the second with the third. `zip` forms those pairs, and `for p0, p1` unpacks each pair. Three waypoints produce two displacements. `enumerate` supplies each index and value. The list comprehension constructs the same ordered displacements as the loop.

2.3 Check the function contract and its results

A **contract** states accepted inputs, the meaning of the result and the handling of invalid input. Week 6 calls an agreed request and answer an **interface**. The following original practice function accepts two tuple/list points and returns their Euclidean distance. It does not print, save a file or change the caller's input. Type annotations describe intended types but do not perform validation.

Validation checks the actual values before arithmetic. The supplied `_point_pair` in `learning_geometry.py` is the complete original Week 2 helper. It checks the container, coordinate count, coordinate types, conversion and

finite values. The separate Boolean check prevents a truth value from being treated as a coordinate. Invalid values raise `PathInputError`, the practice module's named `ValueError`. The helper remains available in the file rather than being repeated here.

Code – the same file, original distance function

```
def distance_example(start_xy, goal_xy) -> float:
    """Return distance for two Python tuple/list points."""
    x0, y0 = _point_pair(start_xy)
    x1, y1 = _point_pair(goal_xy)
    result = math.hypot(x1 - x0, y1 - y0)
    if not math.isfinite(result):
        raise PathInputError("distance must be finite")
    return result
```

Both point checks finish before the coordinate differences are calculated. `math.hypot` calculates the Euclidean distance, and the final check rejects a non-finite result. The `return` supplies one Python float in the input coordinate units.

Run – Anaconda Prompt, stay in week02

```
python function_calls.py
```

Code – function_calls.py, original call forms

```
from learning_geometry import distance_example

print(distance_example((0.0, 0.0), (3.0, 4.0)))
print(distance_example(goal_xy=(3.0, 4.0), start_xy=(0.0, 0.0)))
pair = ((0.0, 0.0), (3.0, 4.0))
print(distance_example(*pair))
```

Output

```
5.0
5.0
5.0
```

The first call uses positional arguments. The second names the parameters. The third uses `*pair` to unpack two points into two arguments. All three calls reach the same function and return the same distance.

Code – the same file, accepted and rejected inputs

```
from learning_geometry import distance_example, PathInputError

for goal in [(3.0, 4.0), (True, 4.0), (3.0,), (float("nan"), 4.0)]:
    try:
        print(distance_example((0.0, 0.0), goal))
    except PathInputError as error:
        print(error)
```

Output

```
5.0
Boolean is not a coordinate
point needs two coordinates
coordinates must be finite
```

The normal point returns `5.0`. Boolean, a missing coordinate and NaN stop at different guards. An **exception** interrupts the normal calculation and can be handled by the caller. The handler displays each error so the loop can continue. It does not turn an invalid point into a numerical result.

A **test** records an expected behaviour as a call and an assertion. An **assertion** fails when its condition is false. These are the same three complete tests as Week 2, not new student-written tests.

Code – test_learning_geometry.py

```
import pytest
from learning_geometry import distance_example, PathInputError

def test_normal_distance():
    actual = distance_example((0.0, 0.0), (3.0, 4.0))
    assert actual == pytest.approx(5.0, rel=1e-12, abs=1e-12)

def test_zero_length():
    assert distance_example((2.0, 1.0), (2.0, 1.0)) == 0.0

def test_invalid_point():
    with pytest.raises(PathInputError):
        distance_example((0.0, 0.0), (True, 4.0))
```

`pytest.approx` allows the stated floating-point tolerance. `pytest.raises` requires the enclosed call to raise the expected exception. Returning a number instead would fail that check.

Run – Anaconda Prompt, week02

```
python -m pytest -q test_learning_geometry.py
```

Output

```
...                                                                      [100%]
3 passed in 0.06s
```

Three passes establish these three cases, not every possible input. The elapsed time depends on the computer.

A wrong result differs from an unfinished function

Week 2's deliberate counterexample adds horizontal and vertical travel instead of calculating the direct distance. It is supplied as `example_wrong_distance.py`.

Code – example_wrong_distance.py

```
import pytest

def wrong_distance(start, goal):
    return abs(goal[0] - start[0]) + abs(goal[1] - start[1])

def test_wrong_distance():
    actual = wrong_distance((0.0, 0.0), (3.0, 4.0))
    assert actual == pytest.approx(5.0, rel=1e-12, abs=1e-12)
```

Run – Anaconda Prompt, week02

```
python -m pytest -q example_wrong_distance.py -x
```

Output

```
E       assert 7.0 == 5.0 ± 5.0e-12
E
E         comparison failed
E         Obtained: 7.0
E         Expected: 5.0 ± 5.0e-12
```

The function returns `3+4=7`, but the test expects the direct distance `5.0`. The failed assertion identifies a wrong calculation, not an installation problem. An `E` prefix introduces failure detail. `-x` stops at the first failure. By contrast, `NotImplementedError` identifies an unfinished body. In the earlier focused test commands, `-k` selected test names and `deselected` counted tests not selected, not failures.

Use the original tolerance comparisons

For finite values, `np.isclose(actual, expected, rtol=..., atol=...)` uses

$$|\text{actual} - \text{expected}| \leq \text{atol} + \text{rtol}\,|\text{expected}|.$$

The absolute tolerance **atol** has the result's units. The relative tolerance **rtol** scales with the reference value. Near zero, the absolute allowance matters because the relative contribution becomes small or zero.

Run – Anaconda Prompt, week02

```
python tolerance_examples.py
```

Code – tolerance_examples.py, original finite comparisons

```
import numpy as np

actual = np.array([5.000004, 5.000020])
print(np.isclose(actual, 5.0, rtol=1e-6, atol=1e-8))

near_zero = np.array([5e-10, 5e-8])
print(np.isclose(near_zero, 0.0, rtol=1e-6, atol=1e-9))
```

Output

```
[ True False]
[ True False]
```

The first allowance is `0.00000501`. The first difference passes and the second fails. For the zero reference, only the absolute allowance `1e-9` remains. Changing a tolerance must not conceal an incorrect formula.

Code – the same file, shape and whole-array comparison

```
expected = np.array([7.0, 10.0])
actual = np.array([7.0 + 1e-13, 10.0])
assert actual.shape == expected.shape
print(np.allclose(actual, expected, rtol=1e-12, atol=1e-12))
# True: every element passes the numerical comparison
```

Output

```
True
```

`np.allclose` returns one Boolean for the entire comparison. The shape assertion checks the intended representation separately. `pytest.approx` accepts a finite result within either its absolute or relative allowance. Unlike `np.isclose`, it does not add the two allowances.

3 Read NumPy arrays by shape and axis

Review connection: Week 2, Sections 2.4–2.5.

The original NumPy examples are collected in `week02/array_examples.py`. They use the original `learning_geometry.py` functions. The complete array-validation helper is supplied in that module, as in Week 2. No Assignment TODO must be completed first.

3.1 Shape identifies the coordinate and waypoint axes

NumPy stores numerical values in arrays. **Shape** gives the length of each dimension, and **dtype** describes the element representation. An **axis** is one array dimension. The program must still state what its positions mean.

Run – Anaconda Prompt, stay in week02

```
python array_examples.py
```

Code – array_examples.py, original path array

```
# Fresh Python session
import numpy as np
path = np.asarray(
    [(0.0, 0.0), (3.0, 0.0), (3.0, 4.0)],
    dtype=np.float64,
)

print(path.ndim)
print(path.shape)
print(path[1, 0])
```

Output

```
2
(3, 2)
3.0
```

The array has two axes. Its three rows are waypoints and its two columns are coordinates. `path[1, 0]` selects the second waypoint's first coordinate, `3.0`. `float64` is the numerical element representation, not the array shape.

Use the original representation table to identify an axis before indexing or reducing it.

Representation	Shape	Axis meaning
point	`(2,)`	axis 0: x and y
path	`(N,2)`	axis 0: waypoint, axis 1: coordinate
batch	`(B,N,2)`	axis 0: path, axis 1: waypoint, axis 2: coordinate

In the batch, `B` paths each contain `N` two-coordinate waypoints. A numerical shape alone does not supply these meanings.

3.2 A view may share data with its source

Code – the same file, original one-row view

```
path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
middle_view = path[1:2]
middle_view[0, 0] = 99.0
print(path)
```

Output

```
[[ 0.  0.]
 [99.  0.]
 [ 3.  4.]]
```

The slice `1:2` retains one row. Writing through `middle_view` changes the original path. A **view** shares storage, while an explicit **copy** has independent storage. The earlier expressions `path[1]`, `path[1:2]` and `path[1:2].copy()` have shapes `(2,)`, `(1,2)` and `(1,2)` respectively.

The next original check uses a separate input and compares it with its saved copy after the function call.

Code – the same file, original input-ownership check

```
from learning_geometry import distance_array_example

candidate = np.array([[0.0, 0.0], [3.0, 4.0]])
snapshot = candidate.copy()
result = distance_array_example(candidate[0], candidate[1])
assert np.isclose(result, 5.0, rtol=1e-12, atol=1e-12)
assert np.array_equal(candidate, snapshot)
print(result)
```

Output

```
5.0
```

The numerical assertion checks distance. `np.array_equal` checks that the input remains unchanged. Numerical correctness and input ownership are different conditions. Week 3 calls multiple references to the same mutable object **aliasing**. A slice may instead be a different array object sharing that object's data.

3.3 Broadcasting produces the original interpolation samples

Interpolation places samples between the given endpoints. It does not plan a detour or assign movement times. Week 2 uses

$$p(\alpha) = (1 - \alpha)p_{\text{start}} + \alpha p_{\text{goal}}, \qquad 0 \le \alpha \le 1.$$

Broadcasting compares dimensions from the right. Each pair must match or have length one. `None` inserts an axis of length one, allowing each interpolation weight to apply to both coordinates.

Code – the same file, original five-sample calculation

```
# Continue with numpy imported; this is the numerical kernel.
start = np.asarray((0.0, 0.0), dtype=np.float64)  # (2,)
goal = np.asarray((3.0, 4.0), dtype=np.float64)   # (2,)
alpha = np.linspace(0.0, 1.0, 5)[:, None]        # (5, 1)

points = start[None, :] + alpha * (goal - start)[None, :]
points[0] = start
points[-1] = goal
print(points.shape)
print(points)
```

Output

```
(5, 2)
[[0.   0.  ]
 [0.75 1.  ]
 [1.5  2.  ]
 [2.25 3.  ]
 [3.   4.  ]]
```

The five rows include both endpoints. The two explicit assignments preserve the normalized endpoints exactly without changing the input arrays. The result's units are the same as the endpoints.

The supplied `interpolate_example` in `learning_geometry.py` adds Week 2's existing point and sample-count checks to this calculation. A sample count must be an integer at least two and must not be Boolean. The original decimal-coordinate call checks the exact endpoint contract separately from interior-point tolerances.

Code – the same file, original decimal endpoints

```
import numpy as np

from learning_geometry import interpolate_example

start = np.array([1.0, 1.0])
goal = np.array([0.1, 0.1])
points = interpolate_example(start, goal, 3)
assert points.shape == (3, 2)
assert np.array_equal(points[0], start)
assert np.array_equal(points[-1], goal)
print(points)
```

Output

```
[[1.   1.  ]
 [0.55 0.55]
 [0.1  0.1 ]]
```

The first and last rows equal the input endpoints. More samples describe the same straight segment more densely. They do not establish collision avoidance or change the route.

3.4 Preserve the meaning when moving from a path to a batch

A path's length is the sum of distances between adjacent waypoints. It is not generally the direct distance between its first and last point. The supplied `path_loop_example` applies the existing distance function to those adjacent pairs.

Code – array_examples.py, original segment and path comparison

```
import numpy as np
from learning_geometry import distance_array_example, path_loop_example

path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
print(distance_array_example(path[0], path[-1]))
print(path_loop_example(path))
```

Output

```
5.0
7.0
```

The direct segment is `5.0`. The path first travels three units and then four, so its length is `7.0`. Both numbers are correct for their respective questions.

Code – learning_geometry.py, original array calculation

```
def path_array_example(path):
    """Kernel for a validated, normalized float64 array (N, 2)."""
    displacement = path[1:, :] - path[:-1, :]
    squared = displacement * displacement
    segment_lengths = np.sqrt(np.sum(squared, axis=1))
    result = float(np.sum(segment_lengths))
    if not np.isfinite(result):
        raise PathInputError("path length must be finite")
    return result
```

The shifted slices subtract adjacent waypoints. Squaring the displacements preserves that row arrangement. `axis=1` combines the two coordinate terms for each segment. The square root gives segment lengths, and the final sum gives one total. Reducing the wrong axis would mix separate segments. This kernel assumes an already validated, normalized `float64` array of shape `(N,2)`.

Code – array_examples.py, original loop/array comparison

```
import numpy as np
from learning_geometry import path_array_example, path_loop_example

path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
actual = path_array_example(path)
expected = path_loop_example(path)
assert np.isclose(actual, expected, rtol=1e-12, atol=1e-12)
assert np.isclose(actual, 7.0, rtol=1e-12, atol=1e-12)
print(actual)
```

Output

```
7.0
```

The same input gives the same `7.0` reference within the stated tolerance. Vectorization changes the execution form, not the geometry.

Code – learning_geometry.py, original batch kernel

```
def batch_example(paths):
    """Kernel for a validated, normalized float64 array (B, N, 2)."""
    displacement = paths[:, 1:, :] - paths[:, :-1, :]
    segment_lengths = np.sqrt(np.sum(displacement**2, axis=2))
    lengths = np.sum(segment_lengths, axis=1)
    if not np.all(np.isfinite(lengths)):
        raise PathInputError("batch lengths must be finite")
    return lengths
```

The first axis retains separate paths. `axis=2` combines coordinates within each segment, then `axis=1` combines segments within each path. The result therefore has one length per path. The same validated `float64` precondition applies.

Code – array_examples.py, original two-path input

```
import numpy as np
from learning_geometry import batch_example

paths = np.array([
    [[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]],
    [[1.0, 1.0], [7.0, 1.0], [7.0, 5.0]],
], dtype=np.float64)
print(paths.shape)
lengths = batch_example(paths)
print(lengths.shape)
print(lengths)
```

Output

```
(2, 3, 2)
(2,)
[ 7. 10.]
```

The first path has length $3 + 4 = 7$, and the second has length $6 + 4 = 10$. Their output order matches their input order. This is the original `(B,N,2)` to `(B,)` calculation, not a coordinate-total calculation.

Code – the same file, original consistency checks

```
from learning_geometry import batch_example, path_loop_example

actual = batch_example(paths)
scalar = np.array([path_loop_example(row) for row in paths])
assert actual.shape == (2,)
assert np.allclose(actual, [7.0, 10.0], rtol=1e-12, atol=1e-12)
assert np.allclose(actual, scalar, rtol=1e-12, atol=1e-12)

shift = np.array([10.0, -2.0])
translated = batch_example(paths + shift)
assert np.allclose(translated, actual, rtol=1e-12, atol=1e-12)

snapshot = paths.copy()
_ = batch_example(paths)
assert np.array_equal(paths, snapshot)
permuted = batch_example(paths[::-1])
assert np.allclose(permuted, actual[::-1], rtol=1e-12, atol=1e-12)
print("All four consistency checks passed")
```

Output

```
All four consistency checks passed
```

The checks compare batch and scalar calculations, translate both paths by the same displacement, preserve the input, and reverse the order of the paths. Translation leaves length unchanged. Reordering paths reorders their results. Each assertion checks the relation stated in Week 2 instead of adding a new expected behaviour.

Return to the review folder

```
cd ..
```

The next section reuses Week 3's objects to keep related values and their behaviour together.

4 Use objects to keep data and behaviour together

Review connection: Week 3, Sections 2–6.

4.1 A class defines the behaviour of its instances

A **class** defines a type of object. An **instance** is a particular object created from that class. An **attribute** stores data, and a **method** is a function defined on the class. In an instance method, `self` refers to the instance receiving the call.

The original Week 3 practice files are supplied in `week03`. Run them separately. The first file contains the ordinary `Configuration` class from Week 3 Section 2.1, not the later Assignment implementation.

Code – week03/01_instances.py, class

```
class Configuration:
    def __init__(self, values):
        self.values = tuple(values)

    def dimension(self):
        return len(self.values)
```

Calling `Configuration((0.0, 0.0))` initializes an instance with the supplied coordinates. `__init__` stores the tuple in `self.values`. It initializes the instance rather than returning the new instance itself.

Run – Anaconda Prompt, from week07_review

```
cd week03
python 01_instances.py
```

Code – the same file, original calls

```
q_start = Configuration((0.0, 0.0))
q_goal = Configuration((3.2, 2.0))

print(q_start.values)
print(q_start.dimension())
print(q_start is q_goal)
```

Output

```
(0.0, 0.0)
2
False
```

`q_start.dimension()` calls the method and returns `2`. Writing `q_start.dimension` only retrieves the method. The `is` comparison is false because `q_start` and `q_goal` are different instances. Instance attributes belong to each instance. Class attributes belong to the class. Week 3's `BadPlanningRun` showed why one mutable class-level list cannot represent two independent histories.

4.2 Dataclasses, ownership and independent histories

`dataclasses` is a standard-library module. Its `@dataclass` decorator generates routine methods, including an initializer for declared fields. The next file imports `FrozenInstanceError` and `dataclass` and contains Week 3's separate frozen practice class. Here, `dimension` is a **property**: read `q.dimension`, without parentheses.

Code – week03/02_dataclass_property.py, class

```
@dataclass(frozen=True)
class Configuration:
    values: tuple

    def __post_init__(self):
        owned = tuple(self.values)
        if not owned:
            raise ValueError("values must not be empty")
        object.__setattr__(self, "values", owned)

    @property
    def dimension(self):
        return len(self.values)
```

The generated initializer calls `__post_init__`. It makes an owned tuple and rejects an empty sequence. `object.__setattr__` stores the normalized tuple during initialization of the frozen object. A type annotation alone does not perform validation. An **invariant** is a condition that must remain true for a valid object. This practice class checks non-emptiness, not the full Assignment numerical contract.

Run – Anaconda Prompt, week03

```
python 02_dataclass_property.py
```

Code – the same file, original caller

```
source = [0.0, 0.0]
q = Configuration(source)
source[0] = 99.0

print("caller list:", source)
print("stored tuple:", q.values)
print("q.dimension:", q.dimension)
print("equal values:", q == Configuration((0.0, 0.0)))
```

Output

```
caller list: [99.0, 0.0]
stored tuple: (0.0, 0.0)
q.dimension: 2
equal values: True
```

The caller's list changes to `[99.0, 0.0]`, but the object's tuple remains `(0.0, 0.0)`. The generated equality method compares field values, so the separately constructed configuration compares equal.

Code – the same file, rejected operations

```
try:
    Configuration(())
except ValueError as error:
    print("empty input rejected:", error)

try:
    q.values = (9.0, 9.0)
except FrozenInstanceError:
    print("field reassignment rejected: FrozenInstanceError")
```

Output

```
empty input rejected: values must not be empty
field reassignment rejected: FrozenInstanceError
```

The empty tuple violates this class's invariant. Ordinary reassignment of a frozen field raises `FrozenInstanceError`. Preventing field reassignment does not prevent mutation inside an array stored in that field.

Run – Anaconda Prompt, week03

```
python array_aliasing.py
```

Code – original Week 3 array_aliasing.py

```
from dataclasses import dataclass
import numpy as np

@dataclass(frozen=True)
class UnsafeConfiguration:
    values: np.ndarray

source = np.array([0.0, 0.0])
q = UnsafeConfiguration(source)
source[0] = 99.0
print(q.values)
```

Output

```
[99.  0.]
```

The field was not reassigned. `source` and `q.values` refer to the same mutable array. This is **aliasing**. Changing the array through one reference is visible through the other. `frozen=True` does not provide deep immutability.

Read-only code – Week 3 Assignment Configuration.as_array

```
def as_array(self) -> NDArray[np.float64]:
    return np.array(self.values, dtype=np.float64, copy=True)
```

This is the original method from Week 3 Section 4.2, not an addition to `02_dataclass_property.py`. It returns an independent NumPy array with `float64` elements. The arrow is a return annotation, not a conversion operation.

Read-only caller – original array_copy.py

```
from ap_week03_planning import Configuration

q = Configuration((1.0, 2.0))
exposed = q.as_array()
exposed[0] = 100.0

print(exposed)
print(q.values)
```

The original caller requires the Week 3 package installation and completed TODO 1–2. It is not one of the standalone review commands. Its first print displays the changed exported array `[100.  2.]`. Its second print displays the unchanged stored tuple `(1.0, 2.0)`. Do not attach this method to the simpler practice class or change its meaning.

Code – week03/practice_history.py, class

```
@dataclass
class PracticeHistory:
    _labels: list[str] = field(default_factory=list, init=False, repr=False)

    def record(self, label: str) -> None:
        if not isinstance(label, str) or not label.strip():
            raise ValueError("label must be a non-empty string")
        self._labels.append(label)

    @property
    def results(self) -> tuple[str, ...]:
        return tuple(self._labels)
```

This is Week 3's original `PracticeHistory`, which records nonblank strings. The file imports `dataclass` and `field` from `dataclasses`. `field(default_factory=list)` creates a different list for each instance. `init=False` excludes the internal list from initializer arguments, and `repr=False` omits it from the generated representation. The underscore marks internal use by convention. It is not access control.

Run – Anaconda Prompt, week03

```
python practice_history.py
```

Code – the same file, original caller

```
first = PracticeHistory()
second = PracticeHistory()
earlier_snapshot = first.results

first.record("first attempt")
print("first history:", first.results)
print("second history:", second.results)
print("earlier snapshot:", earlier_snapshot)

before = first.results
try:
    first.record("   ")
except ValueError as error:
    print("invalid update rejected:", error)

print("history unchanged:", first.results == before)
```

Output

```
first history: ('first attempt',)
second history: ()
earlier snapshot: ()
invalid update rejected: label must be a non-empty string
history unchanged: True
```

`record("first attempt")` changes only `first`'s history and returns `None`. Reading `results` returns a tuple. `earlier_snapshot` holds the history as it was before the append, so it stays empty. The blank label is rejected before mutation, leaving the earlier valid history unchanged. The **snapshot**, stored history and printed output are different things.

4.3 Composition and inheritance express different relationships

Composition means that one object holds other objects as parts of its state. These original Week 3 field declarations show how a `PlanningProblem` connects configurations and bounds. This is a read-only declaration, not a complete class to run.

Read-only code – original PlanningProblem fields

```
@dataclass(frozen=True)
class PlanningProblem:
    problem_id: str
    start: Configuration
    goal: Configuration
    bounds: ConfigurationBounds
```

The start, goal and bounds must have matching dimensions, and both endpoints must lie inside the bounds. Valid parts alone do not guarantee a valid combination.

Inheritance derives a more specific type from a base type. The derived class inherits accessible behaviour and may refine it. Week 3 uses this public error type:

Read-only code – original public error type

```
class PlanningModelError(ValueError):
    """A public planning-domain value violated its contract."""
```

A `PlanningModelError` is a `ValueError`, so a handler for `ValueError` can catch it. This type relationship is different from storing component objects through composition.

5 Choose collections and understand their costs

Review connection: Week 4, Sections 1.2–3.

5.1 Choose the operation before the container

Objects describe individual items. A **collection** organizes several items for ordered access, identifier lookup, duplicate detection or processing requests. The supplied `week04/collections_practice.py` repeats the original Week 4 construction and collection blocks. Its local `ap_week04_scene_data` folder contains the unchanged supplied domain objects, not unfinished Assignment functions.

Run – Anaconda Prompt, from week03

```
cd ..
cd week04
python collections_practice.py
```

Code – original obstacle construction

```
from ap_week04_scene_data import Configuration, Obstacle, PlanningModelError

print("[1.2 Obstacle object]")
obstacle_b = Obstacle(
    obstacle_id="obstacle_b",
    center=Configuration((6.5, 4.0)),
    radius_m=0.6,
)
print("ID:", obstacle_b.obstacle_id)
print("centre:", obstacle_b.center.values)
print("radius (m):", obstacle_b.radius_m)

try:
    Obstacle("invalid", Configuration((6.5, 4.0)), 0.0)
except PlanningModelError as error:
    print(type(error).__name__ + ":", error)
```

Output

```
[1.2 Obstacle object]
ID: obstacle_b
centre: (6.5, 4.0)
radius (m): 0.6
PlanningModelError: radius_m must be a positive finite number
```

`Obstacle` is frozen. Its `center` is a `Configuration`, and `radius_m` must be positive and finite. The supplied `__post_init__` runs during construction and rejects `0.0`. The exception is caused by object validation, not by the surrounding collection.

A **list** is a mutable sequence. It preserves order, allows repeated entries and supports indexing. The original example holds three obstacle objects, then copies the list.

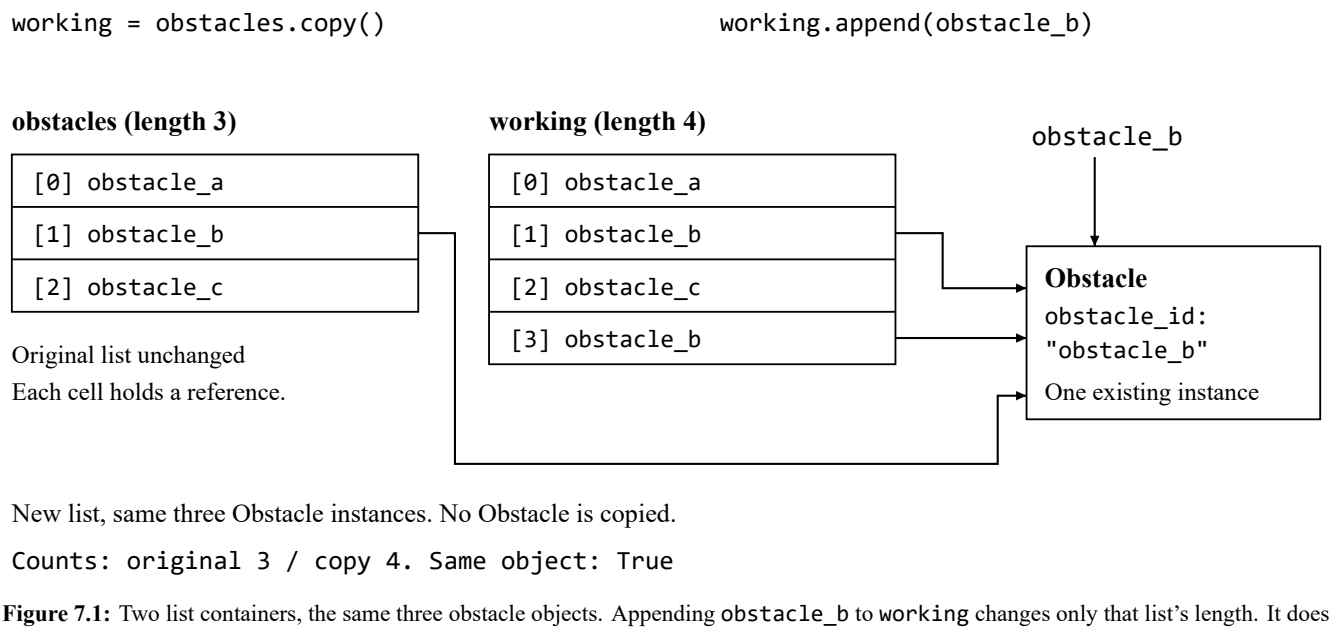


Figure 7.1: Two list containers, the same three obstacle objects. Appending `obstacle_b` to `working` changes only that list's length. It does not duplicate the obstacle. Arrows denote references, not copied values.

Code – the same file, original list example

```
print("[2.1 List]")
obstacle_a = Obstacle("obstacle_a", Configuration((3.0, 2.0)), 0.75)
obstacle_c = Obstacle("obstacle_c", Configuration((8.2, 6.2)), 0.45)
obstacles = [obstacle_a, obstacle_b, obstacle_c]

print(obstacles[0].obstacle_id)
print([item.obstacle_id for item in obstacles[1:]])
working = obstacles.copy()
working.append(obstacle_b)
print("original count:", len(obstacles))
print("copied count:", len(working))
print("same obstacle_b:", working[1] is obstacle_b)
```

Output

```
[2.1 List]
obstacle_a
['obstacle_b', 'obstacle_c']
original count: 3
copied count: 4
same obstacle_b: True
```

`working` is another list, but its existing entries still refer to the same objects. Appending `obstacle_b` changes its length from three to four without changing `obstacles`. The final `True` tests object identity, not equality of numerical values.

A **dictionary** maps keys to values. An obstacle ID selects the associated obstacle without expressing a manual search through the list.

Code – the same file, original dictionary example

```
print("[2.2 Dictionary]")
obstacle_by_id = {}
for item in obstacles:
    obstacle_by_id[item.obstacle_id] = item

print("radius (m):", obstacle_by_id["obstacle_b"].radius_m)
print("same obstacle_b:", obstacle_by_id["obstacle_b"] is obstacle_b)
print("missing ID present:", "missing_id" in obstacle_by_id)
```

Output

```
[2.2 Dictionary]
radius (m): 0.6
same obstacle_b: True
missing ID present: False
```

The retrieved object is the original `obstacle_b`. Membership checks dictionary keys. A repeated key replaces the previous value rather than reporting a duplicate by itself.

A **set** contains distinct values. In this original loop, `seen` records encountered IDs, while `duplicates` keeps each repeated ID once.

Code – the same file, original set example

```
print("[2.3 Set]")
sample = obstacles + [obstacles[0], obstacle_b]
seen = set()
duplicates = set()

for obstacle in sample:
    obstacle_id = obstacle.obstacle_id
    if obstacle_id in seen:
        duplicates.add(obstacle_id)
    else:
        seen.add(obstacle_id)

print(sorted(duplicates))
```

Output

```
[2.3 Set]
['obstacle_a', 'obstacle_b']
```

`sorted` makes the displayed order predictable. Different objects with the same ID also count as duplicates. The criterion is the identifier, not object identity.

5.2 FIFO and LIFO use different deque operations

The requests remain the original Week 4 `PlanningProblem` objects. They use the same bounds and different start/goal configurations. The file creates their parts before creating each request.

Code – the same file, bounds and first request

```
from ap_week04_scene_data import ConfigurationBounds, PlanningProblem

print("[2.4 Request objects]")
bounds = ConfigurationBounds(
    lower=Configuration((0.0, 0.0)),
    upper=Configuration((10.0, 8.0)),
)
request_1 = PlanningProblem(
    problem_id="request_1",
    start=Configuration((1.0, 1.0)),
    goal=Configuration((9.0, 7.0)),
    bounds=bounds,
)
```

Output

```
[2.4 Request objects]
```

Code – the same file, second request and arrival order

```
request_2 = PlanningProblem(
    problem_id="request_2",
    start=Configuration((1.0, 7.0)),
    goal=Configuration((9.0, 1.0)),
    bounds=bounds,
)
requests = [request_1, request_2]
print("arrival order:", [item.problem_id for item in requests])
```

Output

```
arrival order: ['request_1', 'request_2']
```

A **deque** supports insertion and removal at either end. **FIFO** removes the first stored request first. Appending at the right and removing with `popleft()` implements that policy.

Code – the same file, original FIFO example

```
from collections import deque

print("[2.4 FIFO queue]")
request_queue = deque(requests)
current = request_queue.popleft()

print("returned:", current.problem_id)
print("remaining:", [item.problem_id for item in request_queue])
print("original:", [item.problem_id for item in requests])
print("same request:", current is request_1)
```

Output

```
[2.4 FIFO queue]
returned: request_1
remaining: ['request_2']
original: ['request_1', 'request_2']
same request: True
```

`current` is the original `request_1` object, not its printed ID. Removing it from the deque does not remove it from the source list. The remaining deque holds `request_2`.

LIFO removes the last stored request first. The same deque type implements it with right-end `pop()`.

Code – the same file, original LIFO example

```
print("[2.4 LIFO stack]")
stack = deque(requests)
print("returned:", stack.pop().problem_id)
print("remaining:", [item.problem_id for item in stack])
```

Output

```
[2.4 LIFO stack]
returned: request_2
remaining: ['request_1']
```

The selected operations determine the policy. A deque is not limited to FIFO, and the LIFO example also uses a deque, not a list.

5.3 Big-O describes growth, not a stopwatch reading

Computational complexity relates work to input size. For membership, n is the number of stored IDs. For a complete FIFO drain, it is the initial number of requests. **Big-O notation** describes an upper growth class as input size becomes large, omitting constant multipliers and lower-order terms.

The original Week 4 table compares illustrative operation counts at two input sizes. It uses $\log_2 n$, not measured seconds.

Class	Illustrative work	$n = 8$	$n = 16$
O(1)	fixed count independent of n	1	1
O($\log n$)	repeatedly halve a search range	3	4
O(n)	inspect each item once	8	16
O($n \log n$)	n work over logarithmically many levels	24	64
O(n^2)	compare all ordered pairs	64	256

Doubling n adds one halving step, doubles a linear count and quadruples a quadratic count. O(1) does not mean one machine instruction or the fastest result for every small input. Efficient comparison sorting commonly has O($n \log n$) growth. No sorting implementation is required here.

Read each operation and its qualification together in the original cost table. A container does not have one cost for every possible operation.

Operation	Typical cost	Reason or qualification
`items[i]` for a list	O(1)	position directly selects an entry
`target in items` for a list	O(n)	may compare every entry
`key in set` or `dict`	average O(1)	worst case O(n)
`list.pop(0)`	O(n)	later references shift left
`deque.popleft()`	O(1)	removal at the end of a deque
`list.append(x)`	amortized O(1)	occasional resize spread over many appends

Average lookup cost assumes the usual hashing behaviour. Unfavourable collisions can cause O(n) work. **Amortized** cost describes a sequence of operations. An individual list append may resize storage, but the cost spread over many appends is constant per append.

For four requests, front removal from a list shifts $3 + 2 + 1 + 0 = 6$ later references. In general the shifts total $n(n-1)/2$, so draining the list is $O(n^2)$. A deque makes n constant-cost left removals, giving $O(n)$ for the whole drain. One removal and a complete drain are different workloads.

A **benchmark** measures an implementation on a particular input and machine. Week 4 compares an absent target in equivalent list/set data and a complete FIFO drain of equivalent list/deque data. The matching results make those comparisons meaningful. A measured ratio is not a universal speed guarantee.

6 Save values without confusing text and objects

Review connection: Week 4, Section 4.

JSON is a text format for structured values. **Serialization** converts values into a representation for storage or exchange. Parsing reads that representation back. The original `json_practice.py` distinguishes an obstacle object, its Python record and JSON text.

Run – Anaconda Prompt, week04

```
python json_practice.py
```

Code – original JSON text example

```
import json
from ap_week04_scene_data import Configuration, Obstacle

print("[4.1 JSON text]")
obstacle_b = Obstacle("obstacle_b", Configuration((6.5, 4.0)), 0.6)
obstacle_record = {
    "obstacle_id": obstacle_b.obstacle_id,
    "center": list(obstacle_b.center.values),
    "radius_m": obstacle_b.radius_m,
}
text = json.dumps(obstacle_record)
loaded = json.loads(text)

print(text)
print("text type:", type(text).__name__)
print("loaded type:", type(loaded).__name__)
print("same values:", loaded == obstacle_record)
```

Output

```
[4.1 JSON text]
{"obstacle_id": "obstacle_b", "center": [6.5, 4.0], "radius_m": 0.6}
text type: str
loaded type: dict
same values: True
```

`json.dumps` returns a string. `json.loads` returns Python values. The JSON array becomes a list, not a `Configuration`. The recovered dictionary has the same record values but is not yet a validated `Obstacle`.

Code – the same file, original file round trip

```
from pathlib import Path

print("[4.1 JSON file]")
path = Path("practice/obstacle_practice.json")
path.write_text(json.dumps(loaded, indent=2), encoding="utf-8")
reloaded = json.loads(path.read_text(encoding="utf-8"))

print("saved:", path.name)
print("same values:", reloaded == loaded)
```

Output

```
[4.1 JSON file]
saved: obstacle_practice.json
same values: True
```

Generated file

`week04/practice/obstacle_practice.json` contains the record. Open it in the editor to inspect the saved fields. `Path` here is a filesystem path from `pathlib`, not the geometric path class. Reading and writing use the same `encoding="utf-8"`. No encoding algorithm is required.

Week 4 distinguishes three failure stages:

- **Parse error**: invalid JSON text, such as a missing closing brace.
- **Schema error**: fields, types or ranges violate the schema, such as a missing radius or `radius_m: 0`.
- **Domain error**: relationships violate scene rules, such as repeated obstacle IDs or a start outside the bounds.

The short record example uses valid finite values. It does not run the scene loader's schema or domain checks. In Week 4 the loader checks the text and schema before building the scene objects. Successful parsing alone does not establish a usable scene.

Return to the review root before continuing:

```
cd ..
```

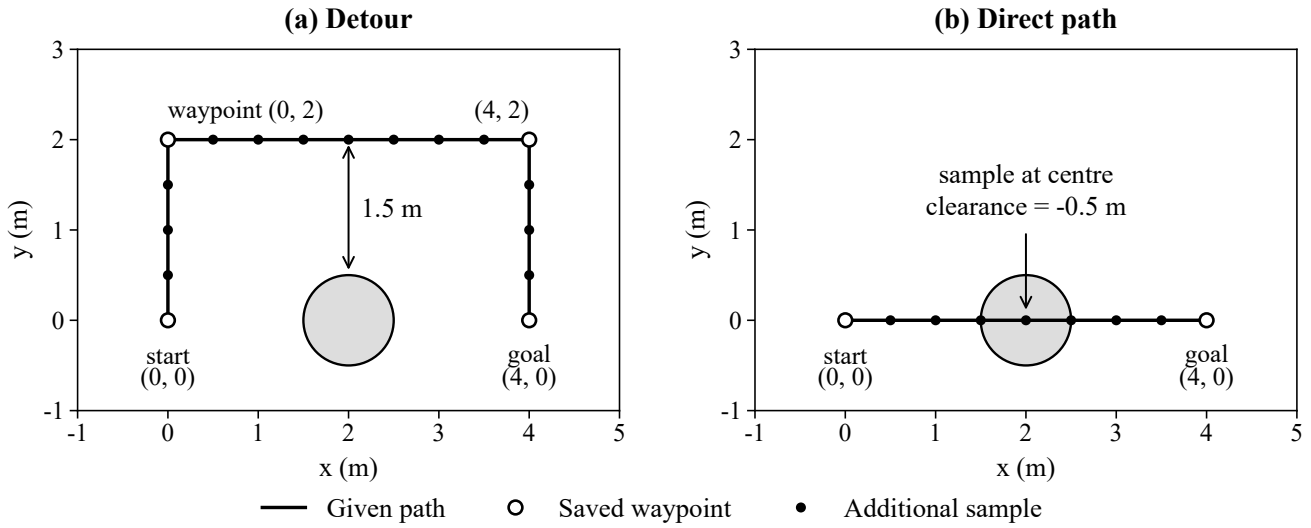
7 Calculate measurements before making a decision

Review connection: Week 5, Sections 1–2.3.

A **parametric model** uses named values to describe a problem while its calculation rules remain the same. We reuse Week 5's start $(0, 0)$, goal $(4, 0)$ and disk with centre $(2, 0)$ and radius 0.5 m. The supplied detour is

$$(0, 0) \longrightarrow (0, 2) \longrightarrow (4, 2) \longrightarrow (4, 0).$$

Its length is 8 m. The direct path connects the same start and goal in 4 m, but passes through the disk.



Fixed obstacle: centre (2, 0) m, radius 0.5 m.

Interpolation spacing: 0.5 m. Both panels use the same workspace and obstacle.

Figure 7.2: The original Week 5 scene and supplied paths. Open circles are saved waypoints. Filled dots are additional checked positions. Clearance is measured to the disk boundary.

The review folder includes the unchanged supplied Week 5 script as `week05/compare_evaluations.py`, together with its original package. Its comparisons use completed support functions and run before the Week 5 Assignment TODOs. No model implementation, package installation or editing of the supplied files is needed here. All commands below run from `week07_review` in the existing Week 6 environment.

7.1 Compare checking methods on the same path

Interpolation calculates intermediate positions between known endpoints. It does not find a route around the obstacle. The `waypoints` method checks only saved waypoints. The `interpolated` method adds check points along each straight segment, using `resolution_m` as the maximum spacing.

For sample position s, obstacle centre o and radius r, clearance is

$$d(s) = \|s - o\|_2 - r.$$

The reported clearance is the minimum over the checked samples. It is positive outside the disk, zero on the boundary and negative inside it.

Code – compare_evaluations.py, supplied measurement function

```
def measurements(case, path, method="interpolated", resolution_m=0.5):
    samples = sample_path(path, resolution_m=resolution_m, method=method)
    metrics = path_metrics(path, samples, case["obstacle"])
    return metrics, len(samples)
```

`sample_path` returns the sample array. `path_metrics` receives the path, samples and obstacle and returns a measurements dictionary. The function returns that dictionary and the number of samples. It does not print them.

Code – the same file, original scene preparation

```
case = supplied_case()
detour = case["path"]
```

`supplied_case()` returns the original scene objects. `case["path"]` is the four-waypoint detour. The following part of the same script builds the direct path from the existing endpoint objects and calls the measurement function twice.

Code – the same file, method comparison

```
direct = Path([case["start"], case["goal"]])

for method in ("waypoints", "interpolated"):
    metrics, count = measurements(case, direct, method=method)
```

Run – Anaconda Prompt, week07_review

```
python week05/compare_evaluations.py method
```

Output

```
path=direct; resolution_m=0.500000
method       sample_count length_m clearance_m
waypoints    2            4.000000 1.500000
interpolated 9            4.000000 -0.500000
```

Both rows have length 4 m. The two saved endpoints are each 1.5 m outside the disk. The nine interpolated samples include its centre, where clearance is -0.5 m. Only the checked positions changed. Neither method moved the original path. An unchecked part of a segment may still violate a condition, so sampled checking is not a physical-safety certificate.

7.2 Compare a measurement with its acceptance limit

A **constraint** is a condition the path must satisfy. Week 5 uses clearance $C \geq c_{\min}$ and length $L \leq L_{\max}$. A **constraint margin** chooses the subtraction direction so an acceptable value is nonnegative:

$$m_C = C - c_{\min}, \qquad m_L = L_{\max} - L.$$

A positive margin leaves room, zero lies on the accepted boundary and a negative margin identifies a violated condition. Both margins have unit metres.

Code – compare_evaluations.py, original margin and decision function

```
def margins_and_decision(metrics, max_length_m=9.0):
    margins = constraint_margins(
        metrics["length_m"], metrics["clearance_m"],
        min_clearance_m=0.5, max_length_m=max_length_m,
    )
    feasible = all(value >= 0.0 for value in margins.values())
    return margins, feasible
```

`constraint_margins` returns the two margins in a dictionary. `all(...)` is `True` only when both signs pass. The function returns the dictionary and the Boolean `feasible`.

Keep the detour's measured length 8 m and clearance 1.5 m. Keep minimum clearance 0.5 m. Change only the maximum length to 9, 8, then 7 m, exactly as in Week 5.

Code – the same file, limits comparison

```
metrics, _ = measurements(case, detour)

for maximum in (9.0, 8.0, 7.0):
    margins, feasible = margins_and_decision(metrics, max_length_m=maximum)
```

Run – Anaconda Prompt, week07_review

```
python week05/compare_evaluations.py limits
```

Output

```
path=detour; method=interpolated; resolution_m=0.500000
min_clearance_m=0.500000
max_length_m length_m clearance_m length_margin_m feasible
9.000000     8.000000 1.500000    1.000000        True
8.000000     8.000000 1.500000    0.000000        True
7.000000     8.000000 1.500000    -1.000000       False
```

The path and measurements stay unchanged. The length margin becomes 1, 0, then -1 m, while the clearance margin remains 1 m. Relaxing a limit changes what we accept, not the path. Compare stored values before formatting them for display. A valid input that fails a constraint returns `feasible=False`. An invalid input raises `PlanningModelError` instead.

8 Generate, shorten and select the original Week 6 routes

Review connection: Week 6, Sections 1 and 3–4.1.

Week 6's point starts at $(0, 0)$ and ends at $(6, 0)$. The two disks have centres $(2, 0.4)$ and $(4, -0.4)$ and radius 0.6 m. Each segment must keep at least 0.25 m clearance, and the final route must be no longer than 6.8 m. An **objective** tells us which accepted result to prefer: the smallest length. A constraint decides whether a candidate enters that comparison.

As in Week 6 Section 2.1, the supplied `continuous_segments` checker checks the closest position over each complete segment and obstacle pair. The Week 6 results below therefore use whole-segment checking, not Week 5's sampled positions. Its geometry is supplied, not a new implementation task.

The bend heights a and b define the original route

$$(0, 0) \to (0, a) \to (2, a) \to (4, b) \to (6, b) \to (6, 0).$$

The supplied constructor removes consecutive duplicate points when a height is zero. The following review keeps the original Week 6 examples and contracts. It does not provide a new implementation of its three Assignment functions.

8.1 Two grid axes produce combinations, not paired heights

A **grid search** tries regularly spaced parameter values. Every value of a combines with every value of b. The axes are bend heights, not the route's x positions.

`np.linspace(low, high, count)` includes both endpoints. For $[-1, 1]$ with count 3, each axis contains `[-1.0, 0.0, 1.0]`. There are $3 \times 3 = 9$ combinations. Week 6's `generate_route_grid` visits a in the outer loop and b in the inner loop. For indices `i, j`, it calls `problem_at_bends(float(a), float(b), settings)` and stores the resulting `RouteProblem` under `f"grid_{i}_{j}"`. It returns the new ID-to-problem dictionary, not a list of coordinate pairs.

The original grid comparison keeps the scene, limits, checker and shortening rule fixed. Both axes cover $[-1.6, 1.6]$ m. Read the candidate counts and the selected result together.

Count per axis	Candidates	Accepted after shortening	Selected result
9	81	0	`None`
17	289	1	`grid_5_11`, about 6.509 m

Spacing changes from 0.4 to 0.2 m. The finer grid includes $a \approx -0.6$, $b \approx 0.6$, which the coarse grid misses. Finding none in 81 trials does not prove that no route exists. The winner is shortest among these generated and improved candidates, not among every possible route. Grid refinement changes candidate routes. Interpolation adds measurement points to an existing route.

8.2 Shortening changes the actual route

For $a = -0.6$, $b = 0.6$, the initial six-point route is about 7.532 m long. Removing the two outer corners gives the four-point result of about 6.509 m. The supplied checker must approve each new direct connection.

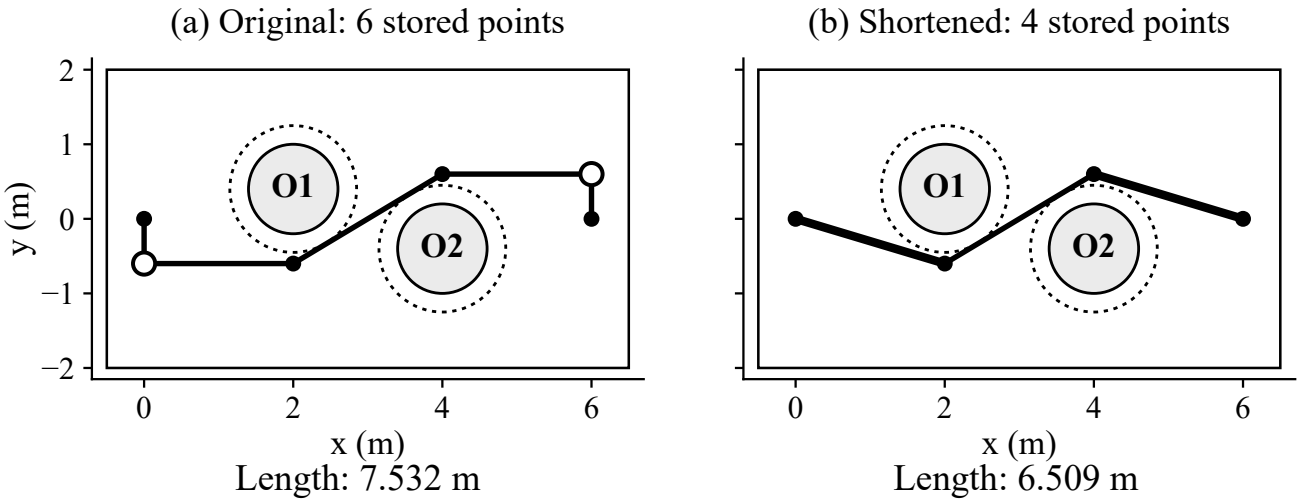


Figure 7.3: The original Week 6 candidate before and after safe shortening. The two outer corners are removed. The middle bends remain because their direct replacements violate clearance.

A **shortcut** removes an internal point and directly connects its neighbours. Its saving is the two old segment lengths minus the new length. Removing $(0, -0.6)$ saves $0.6 + 2 - \sqrt{2^2 + 0.6^2} \approx 0.512$ m. The original table compares all four deletions in the first pass. Values are rounded in metres.

Remove point	New connection	Saving (m)	Safe?
$(0, -0.6)$	$(0, 0) \to (2, -0.6)$	0.512	Yes
$(2, -0.6)$	$(0, -0.6) \to (4, 0.6)$	0.156	No
$(4, 0.6)$	$(2, -0.6) \to (6, 0.6)$	0.156	No
$(6, 0.6)$	$(4, 0.6) \to (6, 0)$	0.512	Yes

The two safe deletions save exactly the same length. The tie rule removes $(0, -0.6)$ first because it occurs first in the current path sequence.

After deletion, compare again. The neighbours of $(2, -0.6)$ are now $(0, 0)$ and $(4, 0.6)$. Its proposed saving changes from 0.156 to 0.376 m, but the new connection is still unsafe. The second pass removes $(6, 0.6)$. Neither remaining internal point can then be removed safely.

At each pass, choose the safe deletion with the largest saving strictly above 10^{-12} m. On an exact tie, keep the first qualifying point in the current path order. This is **greedy** local improvement: choose the best permitted change now, apply it, then repeat. It does not examine every deletion sequence and is not guaranteed to find the best route in every problem.

Preserve start, goal and the original path. Week 6's `shorten_route` modifies a new working list and returns a new `Path`. Apply the final length limit after improvement. Rejecting the initial 7.532 m route first would lose its accepted 6.509 m result.

8.3 Use the provided evaluations to select an accepted ID

The supplied `evaluate_candidates(candidates, evaluator)` returns a dictionary `results` under the same candidate IDs. Each value is a `CandidateResult`. Its `report` holds an `EvaluationReport`, or `None` when unavailable. Its `error` holds an error message, or `None` when evaluation completed without an error. A completed evaluation can still reject a route with `report.valid=False`.

`select_shortest(results)` ignores errors, missing reports and `report.valid=False`. Among accepted reports, return the ID with the smallest `report.metrics.length_m`. Keep the first on an exact length tie. Return `None` if none is accepted.

The original comparison uses a 6.9 m length limit so both detours pass. The main application still uses 6.8 m. Clearance remains at least 0.25 m. Read the rows in order and track the best accepted ID.

ID	Length (m)	Clearance (m)	Accepted?	Best ID so far
`long`	6.869	0.337	Yes	`long`
`blocked`	6.000	−0.200	No	`long`
`short`	6.509	0.257	Yes	`short`

The shortest row, `blocked`, is rejected because it fails clearance. `short` replaces `long` because both pass and it is shorter. The returned value is `"short"`, not the result object or the number 6.509.

Code – original Week 6 record-reading example

```
record = results["short"]
report = record.report
```

These lines assume the evaluated `results` dictionary from the original Week 6 comparison. They read values and do not print output. `record.error` is `None`, `report.valid` is `True`, and `report.metrics.length_m` is about 6.509 m. This is a reading trace, not a standalone executable file in this review folder.

This is **best-so-far selection**. Start with no accepted ID, skip rejected records and update only when an accepted route is shorter. Running the original Week 6 grid application requires its three Assignment functions to be complete. No new generator, shortener or selector answer is supplied here.

9 Practise with the original Week 3 examples

Review connection: Week 3, Sections 2.1, 4.3–4.4 and 6.2.

This is optional review, not a new Assignment or submission. The supplied `week03` files are the original completed examples. To reconstruct them, copy the entire `week03` folder and work inside that copy. Run the same commands there, so the tests use your reconstructed file. Keep the original folder for comparison.

1. Reconstruct the `Configuration` class from `01_instances.py`: its tuple-valued `values`, initializer and `dimension()` method. Keep the original `q_start` and `q_goal` calls. Run the file and compare its three output lines with Section 4.1.
2. Reconstruct `PracticeHistory` using the original `record(label)` method and `results` property. Keep its string labels, per-instance list and rejected-update rule. Run the original caller, then the existing tests below. No new fields or test cases are required.

The supplied `test_practice_history.py` is the original Week 3 file. Its first test records a label, saves a tuple, records another label and checks both histories:

Code – week03/test_practice_history.py, first test

```
def test_normal_record_keeps_order_and_returns_a_snapshot():
    history = PracticeHistory()
    history.record("first attempt")
    snapshot = history.results
    history.record("second attempt")

    assert snapshot == ("first attempt",)
    assert history.results == ("first attempt", "second attempt")
    assert isinstance(history.results, tuple)
```

The remaining original tests check a rejected non-string, two independent instances and unchanged history after a rejected update. These assertions check public behaviour, not a particular internal implementation.

Run – Anaconda Prompt, from week07_review

```
cd week03
python practice_history.py
python -m pytest -q test_practice_history.py
```

The first command reproduces Section 4.2's five history lines. The test command gives:

Output

```
4 passed
```

Passing these four checks confirms their stated cases. If a reconstruction fails, compare the original input, the state before and after the call, the returned tuple and the assertion. No report, screenshot, additional repository or new submission is required.